The basics of programming in C





```
err = read_and_parse("./environ", ...);
if (err != 0) {
    sprintf(path, "/proc/%s/environ", entry->d_name);
    report_error(path, errno);
    ...
```

```
err = read_and_parse("./environ", ...);
if (err != 0) {
    sprintf(path, "/proc/%s/environ", entry->d_name);
    report_error(path, errno);
    ...
```

Library calls that fail are guaranteed to update `errno` to contain the reason for a failure.

Library calls that succeed are guaranteed neither to set `errno` to zero, nor to keep it unchanged.

```
err = read_and_parse("./environ", ...);
if (err != 0) {
    sprintf(path, "/proc/%s/environ", entry->d_name);
    report_error(path, errno);
    ...
```

Library calls that fail are guaranteed to update `errno` to contain the reason for a failure.

Library calls that succeed are guaranteed neither to set `errno` to zero, nor to keep it unchanged.

```
ssize_t getrandom(void *buf, size_t len, int flags)
{
    // try the fast syscall first
    ssize_t x = SYS_getrandom(buf, len, flags);
    if (x >= 0)
        return x;

    // errno contains ENOSYS
    // use /dev/urandom as a fallback
    int fd = open("/dev/urandom", O_RDONLY);
    …
}
```

A successful call to `getrandom()` may set `errno` to a non-zero value.

```
err = read_and_parse("./environ", ...);
if (err != 0) {
    sprintf(path, "/proc/%s/environ", entry->d_name);
    report_error(path, errno);
    ...
```

Library calls that fail are guaranteed to update `errno` to contain the reason for a failure.

Library calls that succeed are guaranteed neither to set `errno` to zero, nor to keep it unchanged.

```
ssize_t getrandom(void *buf, size_t len, int flags)
{
    // try the fast syscall first
    ssize_t x = SYS_getrandom(buf, len, flags);
    if (x >= 0)
        return x;

    // errno contains ENOSYS
    // use /dev/urandom as a fallback
    int fd = open("/dev/urandom", O_RDONLY);
    …
}
```

A successful call to `getrandom()` may set `errno` to a non-zero value.

Additional reading: vDSO and how Linux implements syscalls like `gettimeofday()`.

```
const char* get_executable_name(const char* path)
{
    char *buffer = malloc(FILEPATH_MAX_SIZE);
    ...
```

```
const char* get_executable_name(const char* path)
{
    char *buffer = malloc(FILEPATH_MAX_SIZE);
    ...
```

Make functions static unless they are part of your public API:

```
static char* get_executable_name(...)
```

Pros:

1. The function will not be added to the table of exports of the executable file.
2. More room for optimisation because the compiler knows all callers of the function.

```
size_t cap = 512, len = 0;
char *buf = malloc(cap);
if (!buf) {
    ...
```

```
size_t cap = 512, len = 0;
char *buf = malloc(cap);
if (!buf) {
    ...
```

```
char *buf = fs_malloc(cap);
```

Avoid accidental complexity. Do not add handling of errors that cannot reasonably occur.

```
char *buffer = malloc(cap + 1);
while ((r = fread(buffer + n, 1, cap - n, file)) > 0)
{
    ...
```

```
char *buffer = malloc(cap + 1);
while ((r = fread(buffer + n, 1, cap - n, file)) > 0)
{
    ...
```

```
char *buffer = fs_malloc(cap + 1);
...
```

Have safety checks and sanity checks, especially if they are cheap to implement.

```
char *buffer = malloc(cap + 1);
while ((r = fread(buffer+n, 1, cap-n, file)) > 0)
{
    n += r;
    if (n == cap)
    {
        cap *= 2;
        buffer = realloc(buffer, cap + 1);
    }
}
```

```
char *buffer = malloc(cap + 1);
while ((r = fread(buffer+n, 1, cap-n, file)) > 0)
{
    n += r;
    if (n == cap)
    {
        cap *= 2;
        buffer = realloc(buffer, cap + 1);
    }
}
```

See /include/uapi/linux/limits.h in the Linux kernel sources:

```
#define ARG_MAX 131072 /* # bytes of args + environ
                          for exec() */
```

```
char *buffer = malloc(cap + 1);
while ((r = fread(buffer+n, 1, cap-n, file)) > 0)
{
    n += r;
    if (n == cap)
    {
        cap *= 2;
        buffer = realloc(buffer, cap + 1);
    }
}
```

See `/include/uapi/linux/limits.h` in the Linux kernel sources:

```
#define ARG_MAX 131072 /* # bytes of args + environ
                          for exec() */
```

It is fine to impose limits that are reasonably big and simplify the implementation.

```
char path[512];

snprintf(path, sizeof(path), "/proc/%s/exe", entry->d_name);
```